

01

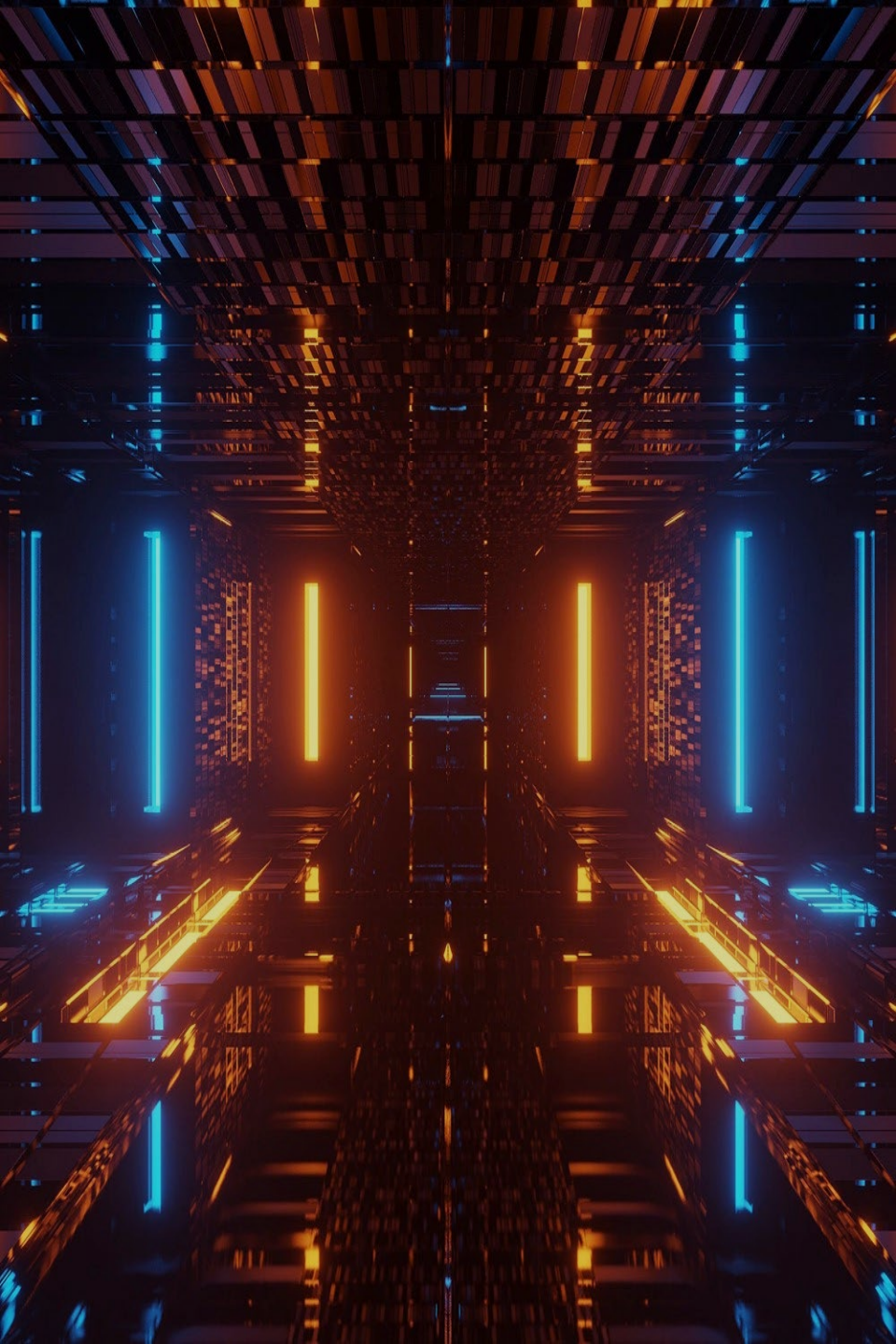
数据科学与大数据技术的数学基础

Mathematical Foundations of Data Science

/02

相似性描述

Data with Distances



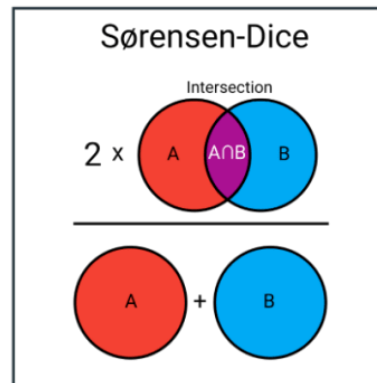
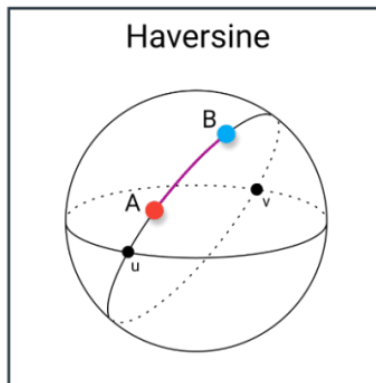
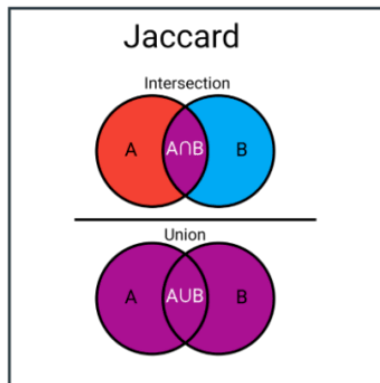
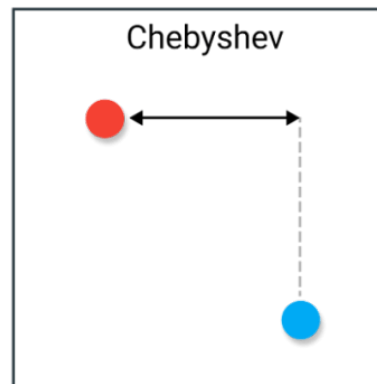
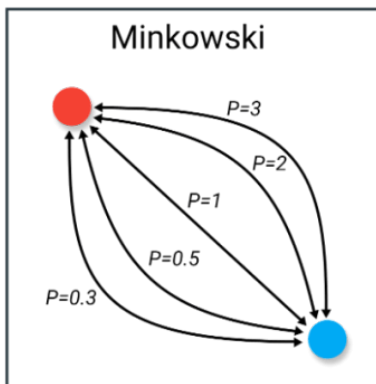
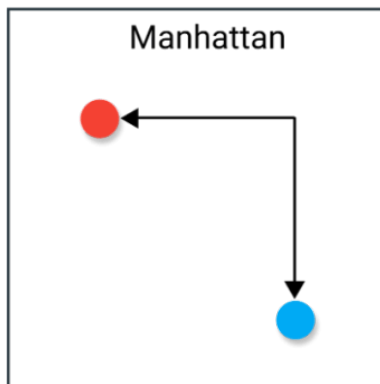
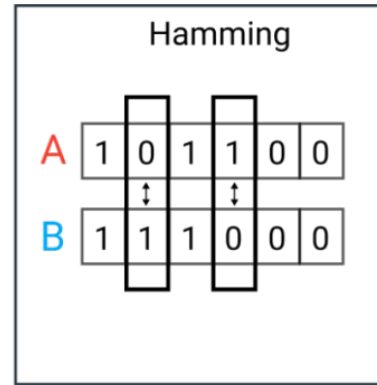
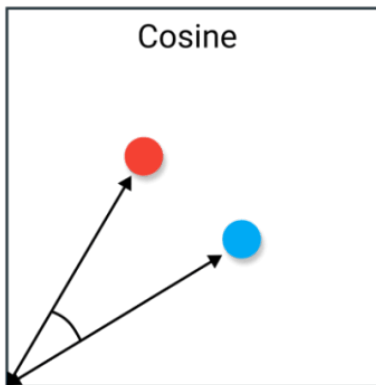
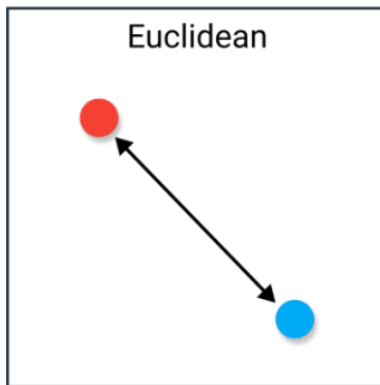
Contents

1. Similarity Search
2. Jaccard Similarity
3. Norm
4. K-d tree
5. Other similarity-related concepts

2.1 Application of Similarity Search in Big Data

- Similar documents, web pages, genomes, etc. (**de-duplication of datasets**, **plagiarism detection** in code/essays, detecting mirror web sites, finding similar genes or sequencing populations of organisms)
- Collaborative filtering: **find similar products** (based on whether the same set of people purchased them) or **individuals** (based on purchase history, demographics, web browsing behavior, etc.).
- **Machine learning/classification**: If we find two similar datapoints, they might have the same label....
- **Combining datasets** (e.g. in astronomy-different telescopes take pictures of the same portions of the sky, maybe in different wavelengths, etc.{very useful to automatically aggregate these datasets}).
- **Super fast labeling**: e.g. in CERN, which bashes particles together and needs to very quickly figure out which new particle traces/trajectories are “interesting” and worth saving, and which trajectories are just boring/common particles.

2.1 9 distance measurement methods



2.2 Jaccard Similarity in Text

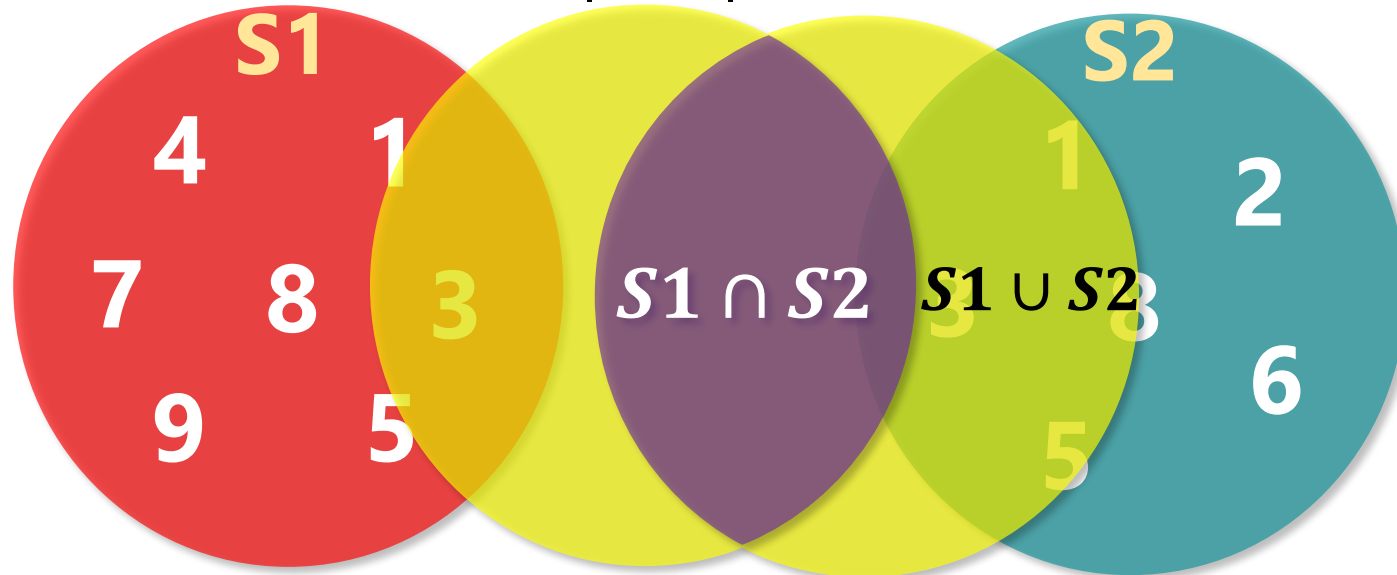
Application of Jaccard similarity

mainly applied to **data clustering**, **comparing the similarity of texts**, used for **text checking and de-duplication**, calculating the distance between objects

$S1 = \{1, 3, 4, 5, 7, 8, 9\}$

$S2 = \{1, 2, 3, 5, 6, 8\}$

$$J(S, T) = \frac{|S \cap T|}{|S \cup T|} = \frac{|S \cap T|}{|S| + |T| - |S \cap T|} \quad J_\delta(S, T) = 1 - \frac{|S \cap T|}{|S \cup T|} = \frac{|S \cup T| - |S \cap T|}{|S \cup T|}$$



2.2 Jaccard Similarity in vector

Application of Jaccard similarity-2

The Jaccard coefficient is well suited for analyzing the similarity between **multiple dimensions** and is also mostly used in recommendation systems to **recommend similar products or businesses to users**.

The Jaccard coefficients reflect the relationship between **two vectors** (whose elements take **the value 0 or 1**). For **$\vec{A}$ and $\vec{B}$ vectors**:

$N_{A_0B_0}$ = the number of the element value in $\vec{A}$ is **0** and in $\vec{B}$ is **0**

$N_{A_1B_0}$ = the number of the element value in $\vec{A}$ is **1** and in $\vec{B}$ is **0**

$N_{A_0B_1}$ = the number of the element value in $\vec{A}$ is **0** and in $\vec{B}$ is **1**

$N_{A_1B_1}$ = the number of the element value in $\vec{A}$ is **1** and in $\vec{B}$ is **1**

$$J(A, B) = \frac{N_{A_0B_0} + N_{A_1B_1}}{N_{A_0B_0} + N_{A_1B_0} + N_{A_0B_1} + N_{A_1B_1}}$$

However, we tend to **ignore** $N_{A_0B_0}$ when $N_{A_0B_0}$ is large or when we do not focus on the case it represents

$$J(A, B) = \frac{N_{A_1B_1}}{N_{A_1B_0} + N_{A_0B_1} + N_{A_1B_1}}$$

2.2 Jaccard Similarity in vector

?

Suppose you are calculating the similarity of two users of a certain web. You can analyze them in terms of gender, region, age, browsing time, etc. Let's further refine these **dimensions**:

{Male, female, younger than 18, 18-40, 40+, browsing time in the morning, browsing time in the middle of the day, browsing time in the afternoon}

The above dimensions are used as a set, and for two users A and B, **the indicator values that meet the above dimensions are set to 1 and the others are set to 0.**

1) User A is a 24-year-old male who likes to use the Internet in 15:00

2) User B is a 15-year-old male who likes to surf the Internet in 13:00

What is the **Jaccard similarity** of these two people?

2.2 Jaccard Similarity in vector

About Jaccard similarity, this PPT references the relevant settings in the MATLAB environment. please note that in practice, **the core of Jaccard is to calculate the degree of similarity of the two parties being compared.**

Jaccard similarity, which we denote by $J(\cdot, \cdot)$, is a distance metric between two sets (or two multisets-e.g. sets where elements are allowed to appear more than once) of objects, S, T :

$$J(S, T) = \frac{|S \cap T|}{|S \cup T|}$$

Equivalently, if we represent our sets (or multisets) S, T as vectors v_S, v_T , with the i th index of the vector $v_S(i)$ equaling the number of times that the i th element is represented in S , the above definition becomes:

$$J(S, T) = J(v_S, v_T) = \frac{\sum_i \min(v_S(i), v_T(i))}{\sum_i \max(v_S(i), v_T(i))}$$

This expression is undefined if S, T are both the empty set, in which case we can define the distance to be 0.

Jaccard similarity works quite well in practice, especially for sparse data. For example, if we represent documents in terms of the multiset of words they contain, then the Jaccard similarity between two documents is often a reasonable measure of their similarity. Similarly, to estimate similarity between individuals, an online marketplace like Amazon, might represent people as multisets of items purchased, etc., or movies reviewed, etc., and use Jaccard similarity.

PS Cosine Distance

Cosine Distance

Cosine Distance can also be called **Cosine Similarity**. The **cosine** of an angle in geometry can be used to measure the **difference between the directions of two vectors**, a concept borrowed in machine learning to measure the difference between sample vectors. Compared with the distance measure, cosine similarity focuses more on the difference between two vectors in terms of direction rather than distance or length.

$$\cos(x, y) = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}} = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \cdot \|\vec{y}\|}$$

The cosine similarity of two vectors with **identical** directions is **1**, while the cosine similarity of two vectors that are **opposite** to each other is **-1**.

Remember the user similarity example in **Jaccard distance**? Cosine similarity is also commonly used in **product recommendation systems** or for **text similarity determination**. Let's see how cosine similarity can be used to compare text and user similarity.

Jaccard-text

Cosine-text

Jaccard-user

Cosine-user

Although we say that one of the main disadvantages of cosine similarity is **that it does not consider the size of the vectors, but only their direction**, **in practice, we affect the result of cosine similarity by increasing the length of the vectors in a certain dimension**. For example, if a user likes an item and clicks it twice, $x_i=2$, or if a participle appears m times in the text, $y_i=m$.

Jaccard Similarity VS Cosine Similarity

Text 1: AI is our friend and it has been friendly.

Text 2: AI and humans have always been friendly.

In order to calculate the Jaccard similarity, we first use **Lemmatization** (commonly used in English **Natural Language Processing**), to **replace those words with the same root by their roots**.

S1={AI, is, our, friend, and, it, has, been}

S2={AI, and, human, has, always, been, friend}



Jaccard Similarity: $5/(5+3+2)=0.5$

Jaccard Similarity VS Cosine Similarity

Text 1: AI is our friend and it has been friendly.

Text 2: AI and humans have always been friendly.

Cosine Distance

Cosine Distance can also be called **Cosine Similarity**. The **cosine** of an angle in geometry can be used to measure the **difference between the directions of two vectors**, a concept borrowed in machine learning to measure the difference between sample vectors. Compared with the distance measure, cosine similarity focuses more on the difference between two vectors in terms of direction rather than distance or length.

$$\cos(x, y) = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}} = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \cdot \|\vec{y}\|}$$

Since cosine similarity is calculated using vectors, we must first convert the sentence text into the corresponding vector. The simplest one is to use **TF** (term frequency) and TF-IDF calculated by **bag of words**.

Jaccard Similarity VS Cosine Similarity

Text 1: AI is our friend and it has been friendly.

Text 2: AI and humans have always been friendly.

1. Use bag of words to calculate term frequency. The following figure shows the statistical results of word frequency.

Term Frequency:

Sentence	AI	IS	OUR	FRIEND	AND	IT	HAS	BEEN	HUMAN	ALWAYS
1	1	1	1	2	1	1	1	1	0	0
2	1	0	0	1	1	0	1	1	1	1

2. First **sum** the **squares** of each word frequency, and then **take the square root**. Then **divide** each word frequency by the **L2-norm value of this sentence**.

L2-norm: Sentence 1: 3.3166; Sentence 2: 2.6458

Sentence	AI	IS	OUR	FRIEND	AND	IT	HAS	BEEN	HUMAN	ALWAYS
1	0.302	0.302	0.302	0.603	0.302	0.302	0.302	0.302	0	0
2	0.378	0	0	0.378	0.378	0	0.378	0.378	0.378	0.378

Jaccard Similarity VS Cosine Similarity

Text 1: AI is our friend and it has been friendly.

Text 2: AI and humans have always been friendly.

L2-norm: Sentence 1: 3.3166; Sentence 2: 2.6458

Sentence	AI	IS	OUR	FRIEND	AND	IT	HAS	BEEN	HUMAN	ALWAYS
1	0.302	0.302	0.302	0.603	0.302	0.302	0.302	0.302	0	0
2	0.378	0	0	0.378	0.378	0	0.378	0.378	0.378	0.378

3. Substituting into the **cosine similarity formula**, the numerator is the sum of the **dot products** of each element, and the denominator is the **modulus** of the two sentences and then finding the **product**. Since the word frequency has been normalized in the previous step, the calculation results are as follows:

$$\cos(x, y) = \frac{\vec{x} \cdot \vec{y}}{|\vec{x}| \cdot |\vec{y}|} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}} = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \cdot \|\vec{y}\|}$$

Cosine Similarity: $(0.302 \times 0.378) + (0.603 \times 0.378) + (0.302 \times 0.378) + (0.302 \times 0.378) + (0.302 \times 0.378) = 0.684$

2.3 Norm

Norm

A function with the concept of "**length**". A measure of the **length or size** of each **vector** in a vector space (or **matrix**).

In **linear algebra**, generalized functional analysis, and related mathematical fields, a **norm** is a function that satisfies **positive definite, homogeneous, and triangular inequalities**.

In linear algebra **X**

- 1) $\|\vec{x}\| \geq 0$ and $\|\vec{x}\| = 0 \leftrightarrow \vec{x} = 0$ for $\vec{x} \in X$ (**positive definite**, *when* $\vec{x} = 0$, $\|\vec{x}\| = 0$)
- 2) $\|c\vec{x}\| = |c|\|\vec{x}\|$ for $c \in \mathbb{R}$ (数域), $\vec{x} \in X$ (**homogeneous**)
- 3) $\|\vec{a} + \vec{b}\| \leq \|\vec{a}\| + \|\vec{b}\|$ for $\vec{a}, \vec{b} \in X$ (**triangular inequalities**)

The meaning of norm

For **scalar quantities**, we can **compare** their sizes intuitively; **vectors and matrices cannot** compare their sizes directly like scalars. **Norm** are used to **compare the size of matrices or vectors** by mapping them to real numbers whose size can be compared.

2.3 Norm

Norm of vectors

indicates the size of the original set

l_0 -norm: If the vector $\vec{x}$ is $(x_1, x_2, \dots, x_n)$, **l_0 norm** of $\vec{x}$ is

$$\|\vec{x}\|_0 = \text{Number of all non-zero elements in } \vec{x}$$

If vector $\vec{a} = (0, 3, 6)$, $\vec{a}$ has 1 element 0 and 2 non-zero elements (3 and 6), then corresponds to l_0 -norm $\|\vec{a}\|_0 = 2$

l_1 -norm: If the vector $\vec{x}$ $(x_1, x_2, \dots, x_n)$, **l_1 norm** of $\vec{x}$ is

$$\|\vec{x}\|_1 = \text{sum of the absolute values of all elements in } \vec{x}$$
$$= |x_1| + |x_2| + \dots + |x_n| = \sum_{i=1}^n |x_i|$$

If vector $\vec{a} = (0, 3, 6)$, corresponds to l_1 -norm $\|\vec{a}\|_1 = |0| + |3| + |6| = 9$

L1 and L2 norm are mainly used as loss functions and regular terms in machine learning

l_2 -norm: for vector $\vec{x}$ $(x_1, x_2, \dots, x_n)$

$$\|\vec{x}\|_2 = \|\vec{x}\|$$

= *sum of the squares of the absolute values of all elements in $\vec{x}$, and then find square root*

$$= \sqrt{\sum_{i=1}^n |x_i|^2}$$

If vector $\vec{a} = (0, 3, 6)$, corresponds to l_2 -norm $\|\vec{a}\|_2 = \sqrt{0 + 3^2 + 6^2} = \sqrt{45}$

l_∞ -norm: If the vector $\vec{x}$ is $(x_1, x_2, \dots, x_n)$

$$\|\vec{x}\|_\infty = \text{Max of absolute values all elements in } \vec{x}$$
$$= \max(|x_1|, |x_2|, \dots, |x_n|)$$

If vector $\vec{a} = (0, 3, 6)$, to L_∞ -norm $\|\vec{a}\|_\infty = 6$

l_p -norm: for vector $\vec{x}$ $(x_1, x_2, \dots, x_n)$

$$\|\vec{x}\|_p = \sqrt[p]{|x_1|^p + |x_2|^p + \dots + |x_n|^p}$$

= *Unifying the l_1 , l_2 and l_∞ norm into one framework system*

$$= \sqrt[p]{\sum_{i=1}^n |x_i|^p}$$

2.3 Manhattan Distance/ l_1 distance

l_1 -norm

Sum of the absolute values of the elements of the vector

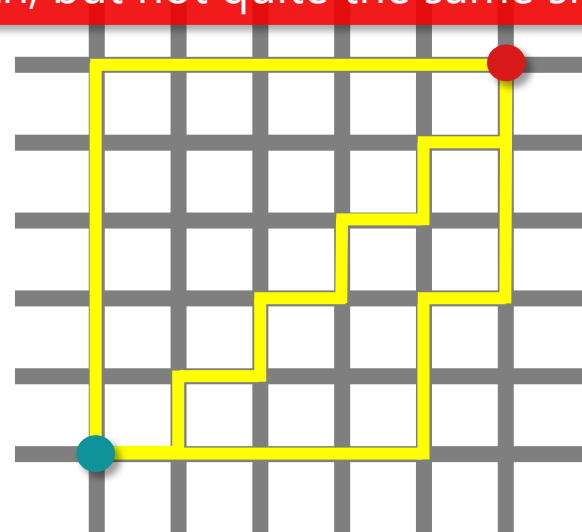
Suppose there is a **vector** $\vec{x}$, the coordinates of vectors $\vec{x}$ is $(x_1, x_2, \dots, x_n)$

Looking at this diagram, notice one

$$\|\vec{x}\|_1 = \sum_i |x_i|$$

l_1 -norm is also called **taxicab-norm** or **Manhattan-norm**. For example, a taxi ride in Manhattan, the car can only travel along the **block road** arranged in **latitude and longitude lines**. The yellow line between the blue starting point and the red endpoint represents the taxicab distance between the two points, that is, the **l_1 distance**, is it very similar to the navigation route in Beijing?

Looking at this diagram, notice one thing, the Manhattan distance (l_1 distance) between two points may have multiple routes with the same length, but not quite the same shape



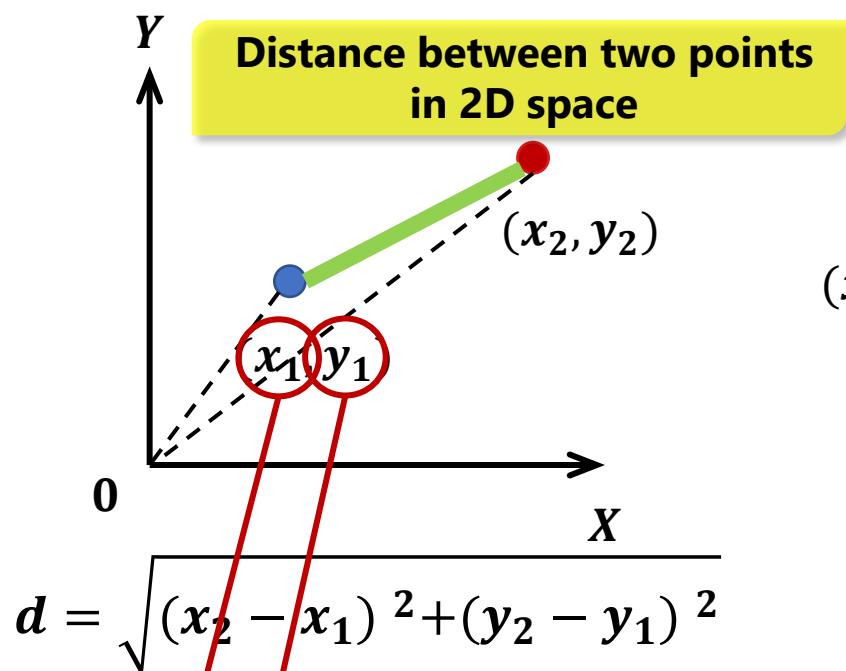
Manhattan distance

$$\|\vec{x}, \vec{y}\|_1 = \sum_i |y_i - x_i|$$

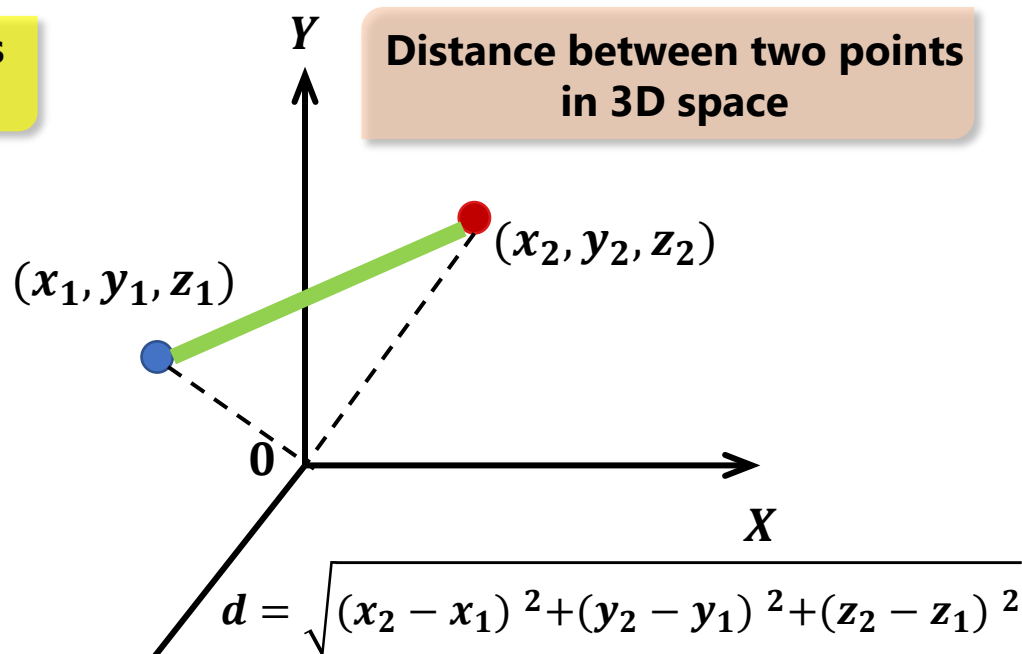
2.3 Euclidean Distance

Euclidean Distance

The **true distance between two points in n-dimensional space**, or the natural length of the vector (i.e., the distance from that point to the origin)



Given datapoints in $\mathbb{R}^n$, Position of point A $= (a_1, a_2, \dots, a_n)$, Position of point B $= (b_1, b_2, \dots, b_n)$, the Euclidean distance metric



$$D_{euclidean} = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

a_i refers to the coordinates of point A in the i -th dimension

2.3 Euclidean Distance/ l_2 distance

Euclidean norm == **Euclidean distance** == **l_2 -norm** == **l_2 distance**

Suppose there is a **vector** $\vec{x}$, the coordinates of vectors $\vec{x}$ is $(x_1, x_2, \dots, x_n)$

$$\|\vec{x}\|_2 = \sqrt{\sum_{i=1}^n (x_i)^2} = \sqrt{(x_1)^2 + (x_2)^2 + \dots + (x_n)^2}$$

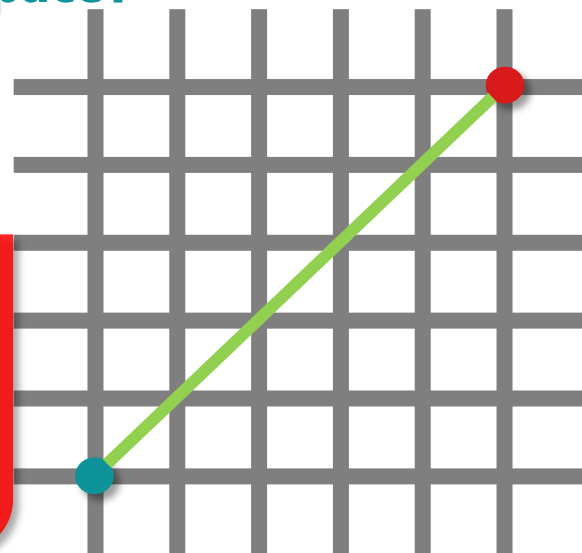
?

If $\vec{x} = [3, 2, 4, 5, 2]$, $\|\vec{x}\|_2$?

Suppose there is **vector** $\vec{x}$ and $\vec{y}$, the coordinates of vectors $\vec{x}$ is $(x_1, x_2, \dots, x_n)$, the coordinates of vectors $\vec{y}$ is $(y_1, y_2, \dots, y_n)$, **L_2 -norm** can be considered as the **distance between 2 points in n-dimensional space**:

$$D_{euclidean}(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

The **Euclidean distance** between two points is shown by the green line, it has only 1 route and the squared term appears in the formula (the derivative is continuous and may be possible to use the triplet of the calculus <derivative, set to zero, solve the equation>)



2.3 Chebyshev Distance/ l_∞ distance

l_∞ -norm Max of absolute values of all elements in the vector

Suppose there is a **vector** $\vec{x}$, the coordinates of vectors $\vec{x}$ is $(x_1, x_2, \dots, x_n)$

$$\|\vec{x}\|_\infty = \max(|x_1|, |x_2|, \dots, |x_n|)$$

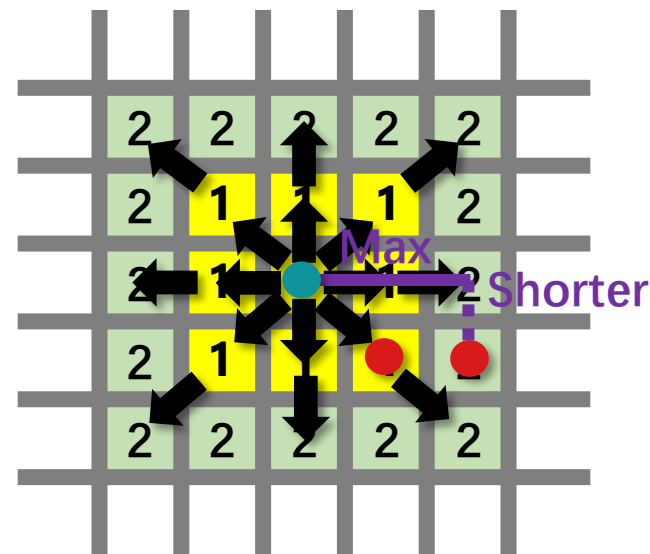
Chebyshev Distance

Chebyshev distance is defined as the **maximum difference between two vectors in any coordinate dimension**.

The Chebyshev distance is often referred to as the **board distance** because the minimum number of moves a **chess king** can make from one square to another is equal to the Chebyshev distance.

$$\|\vec{x}, \vec{y}\|_\infty = \max(|x_i - y_i|)$$

The king can walk diagonally, and the step length counts as 1



In horizontal axis, $|a_1 - b_1| = 2$
In vertical axis, $|a_2 - b_2| = 1$
Chebyshev Distance = **2**

2.3 Minkowski Distance/ l_p distance

l_p -norm a generalized formulation of several distance formulas

Suppose there is a **vector** $\vec{x}$, the coordinates of vectors $\vec{x}$ is $(x_1, x_2, \dots, x_n)$

$$\|\vec{x}\|_p = \sqrt[p]{|x_1|^p + |x_2|^p + \dots + |x_n|^p} = \sqrt[p]{\sum_{i=1}^n |x_i|^p}$$

Chebyshev Distance

$$\|\vec{x}, \vec{y}\|_p = \sqrt[p]{\sum_{i=1}^n (|x_i - y_i|)^p}$$

$P = 1, \|\vec{x}\|_1 = \sum_{i=1}^n |x_i|, \|\vec{x}, \vec{y}\|_1 = \sum_{i=1}^n (|x_i - y_i|) \Rightarrow l_1$ -norm **Manhattan Distance**

$P = 2, \|\vec{x}\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2}, \|\vec{x}, \vec{y}\|_2 = \sqrt{\sum_{i=1}^n (|x_i - y_i|)^2} \Rightarrow l_2$ -norm **Euclidean Distance**

$P \rightarrow \infty, \|\vec{x}\|_\infty = \max(|x_i|), \|\vec{x}, \vec{y}\|_\infty = \max(|x_i - y_i|) \Rightarrow l_\infty$ -norm **Chebyshev Distance**

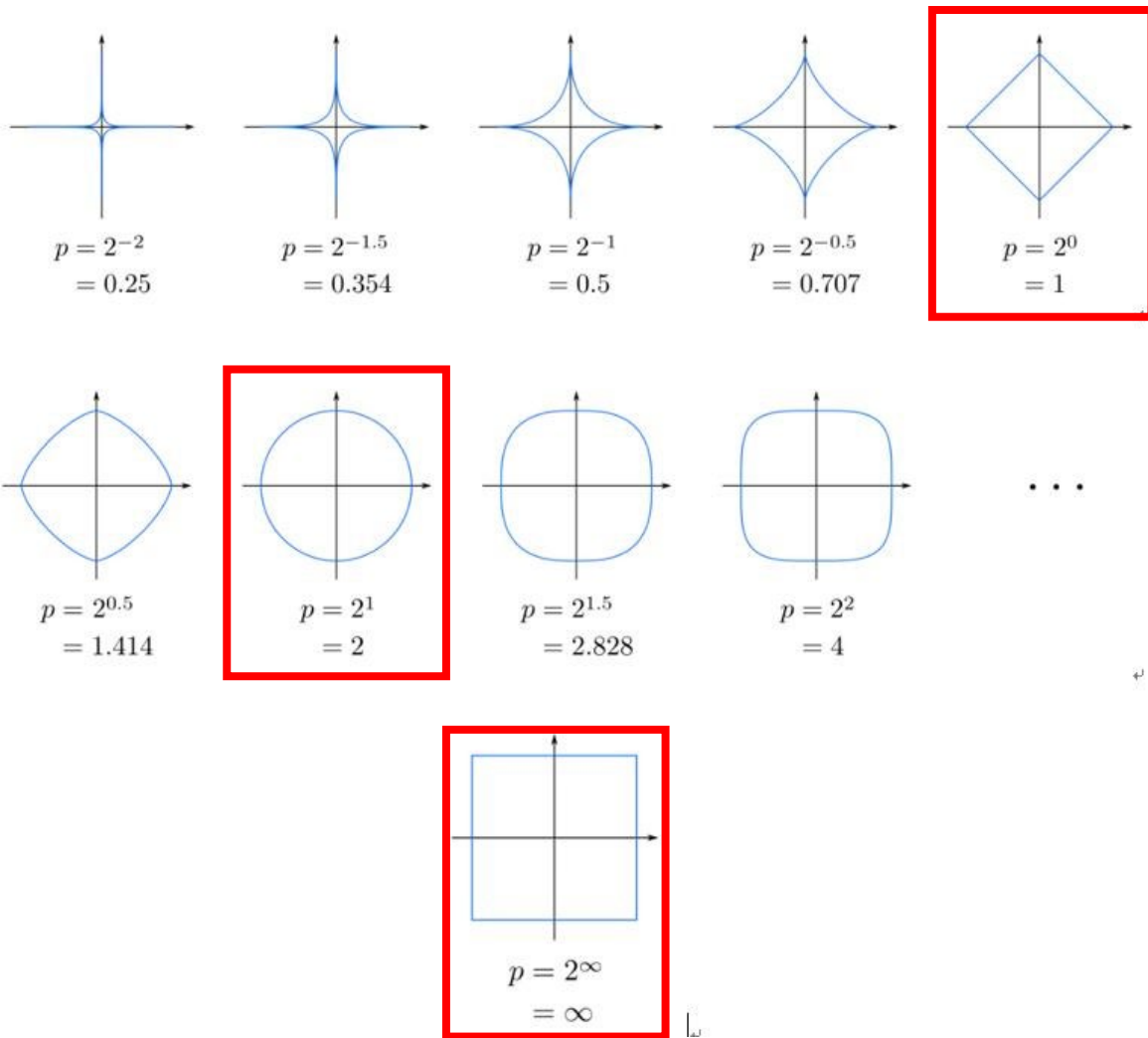
2.3 Minkowski Distance/ l_p distance

l_p -norm

$$\|\vec{x}\|_p = \sqrt[p]{|x_1|^p + |x_2|^p + \dots + |x_n|^p} = \sqrt[p]{\sum_{i=1}^n |x_i|^p}$$

The geometric interpretation of the l_p -norm can be illustrated by the **unit circle**. The unit circle is the **set of infinite vector endpoints** that must satisfy an $\|\vec{x}\|_p = 1$ and start at the origin.

1. l_1 -norm: its unit circle in 2d space is a square
2. l_2 -norm: its unit circle in 2d space is a circular
3. l_∞ -norm: its unit circle in 2d space is also a square
4. l_p -norm: it is a hyperellipse (with congruent axes)



2.3 Matrix Norm

For any matrix $A \in C^{m \times n}$, there exists a real number $\|A\|$ associated with it, satisfying the following properties:

1. Positive Definiteness:

$\|A\| \geq 0$, with equality if and only if $A = 0$.

2. Homogeneity:

For any scalar $\lambda \in C$, $\|\lambda A\| = |\lambda| \|A\|$.

3. Triangle Inequality:

For any matrices $A, B \in C^{m \times n}$, $\|A + B\| \leq \|A\| + \|B\|$.

[Note] All matrix norms satisfy the first three properties above. Such a function $\|A\|$ is called a **matrix norm** on $C^{m \times n}$.

4. Submultiplicativity:

For any matrices $A, B \in C^{m \times n}$, $\|AB\| \leq \|A\| \|B\|$.

[Note] General matrix norms do not necessarily satisfy the submultiplicativity condition.

5. Let $\|\cdot\|_m$ be a matrix norm on $C^{m \times n}$ and $\|\cdot\|_v$ be a vector norm on C^n . If for any $A \in C^{m \times n}$ and $x \in C^n$, the inequality

$$\|Ax\|_v \leq \|A\|_m \|x\|_v$$

holds, then the matrix norm $\|\cdot\|_m$ and the vector norm $\|\cdot\|_v$ are called compatible.

6. Let $\|\cdot\|_m$ be a matrix norm on $C^{m \times n}$. Then there necessarily exists a vector norm on C^n that is compatible with it.

2.3 Matrix Norm

$$\begin{aligned} \text{Tr} &= \text{trace} \\ \text{tr}(A) &= \sum_{i=1}^n a_{ii} \\ &= a_{11} + a_{22} + \dots + a_{nn} \end{aligned}$$

$$1. \|A\|_{m_1} = \sum_{i=1}^m \sum_{j=1}^n |a_{ij}|, \quad m_1 - \text{范数}$$

$$2. \|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2} = \sqrt{\text{tr}(A^H A)}, \quad F - \text{范数}$$

酉不变

$$3. \|A\|_M = \max\{m, n\} \max_{i,j} |a_{ij}|, \quad M - \text{范数或最大范数}$$

$$4. \|A\|_{m_\infty} = n \max_{i,j} |a_{ij}|, \quad m_\infty - \text{范数}$$

$$5. \|A\|_G = \sqrt{mn} \max_{i,j} |a_{ij}|, \quad G - \text{范数或几何平均范数}$$

$$6. \|A\|_1 = \max_j \sum_{i=1}^m |a_{ij}|, \quad 1 - \text{范数或列和范数}$$

$$7. \|A\|_2 = \sqrt{\lambda_{\max}(A^H A)}, \quad 2 - \text{范数或谱范数}$$

$$8. \|A\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|, \quad \infty - \text{范数或行和范数}$$

m_1 - 范数与向量 l_1 - 范数相容; F - 范数和 G - 范数与向量 l_2 - 范数相容; M - 范数分别与向量 l_1 - 范数、 l_2 - 范数、 l_∞ - 范数相容; 而矩阵的 1-范数、2-范数、 ∞ - 范数分别与相应的向量范数相容

2.4 K-d Tree

Classification is an important technique in the field of **data mining**, which discovers a classification model from a set of classified training samples and applies this classification model to classify unknown data.

K-Nearest Neighbor algorithm, KNN

KNN is one of the most straightforward methods used to classify unknown data. It can be seen as: there are so many data that already know the class, when a new data comes in, calculate the distance between this data and each training data in turn, then **pick the K points closest to this data, see what type this K points belong to**, and then use the principle of **minority rule** to classify the new data..

Advantages

- ✓ easy to implement;
- ✓ no need to retrain when new add to the training set;
- ✓ Time and space complexity is linearly related to the size of the training set.

Disadvantages

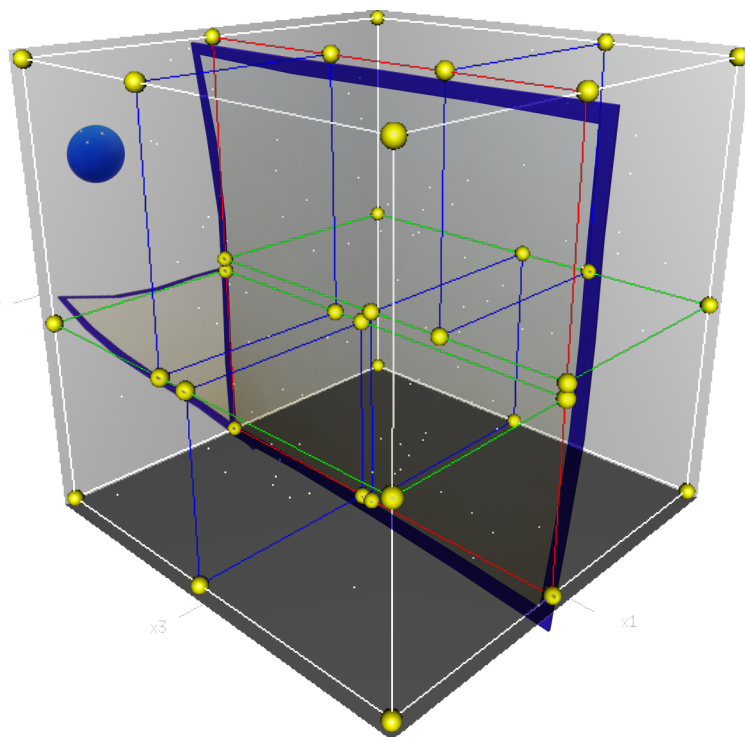
- ✓ slow classification speed (affected by the size of the training set and feature dimension);
- ✓ The classes with more samples are more likely to affect the results;
- ✓ K value is not easy to determine.

2.4 K-d Tree

kd-Tree

A k-dimensional tree is a tree data structure that stores instance points in a k-dimensional space for fast retrieval. K-d trees are **binary trees** that represent a **partition** of a k-dimensional space. Constructing a k-d tree is equivalent to continuously slicing the k-dimensional space with a hyperplane perpendicular to the coordinate axes.

Partition: Suppose there are 300 points in the k-dimensional space, and I input a point **p** in this k-dimensional space, and I want to find the **M points closest to the point p**; then obviously, I have to calculate the distance with these 300 points one by one, and then sort them, that is, "brute force calculation". This is obviously very inefficient, **so we need to divide the 300 points into multiple regions**. I only need to find the nearest M points in the current region and the adjacent regions, not all of them. K-dtree is a partition of the K-dimensional space using a **hyperplane**. The so-called hyperplane is used to **divide the space**, imagine a two-dimensional plane of paper, a knife vertically cut the two-dimensional plane of paper, the knife is the hyperplane.



3D point cloud delineation using kd-tree in **LiDAR SLAM**

2.4 K-d Tree: Building the k-d tree

Input: Unordered point cloud with dimension k

Output: point cloud corresponding to the **k-d tree**

Algorithm.

1. **Initialize the partition axis:** calculate the variance of the data in each dimension, and take the **dimension with the largest variance** as the partition axis, marked as r .
2. **Determine the node:** search the current data by the dimension of the split axis, find the **median data** and put it into the current node.

3. **Divide the double branch:**

Divide the left branch: in the current split axis dimension, all values **less** than the median are divided into the left branch.

Divide the right branch: in the current partition axis dimension, all values **greater than or equal to** the median are divided into the right branch.

4. **Update the partition axis:** $r = (r + 1) \% k$;

5. **Determine the child nodes.**

Determine the left node: perform step 2 in the data of the left branch.

Determine the right node: step 2 in the data of the right branch.

2.4 K-d Tree: Retrieve (data)

Retrieve (data)

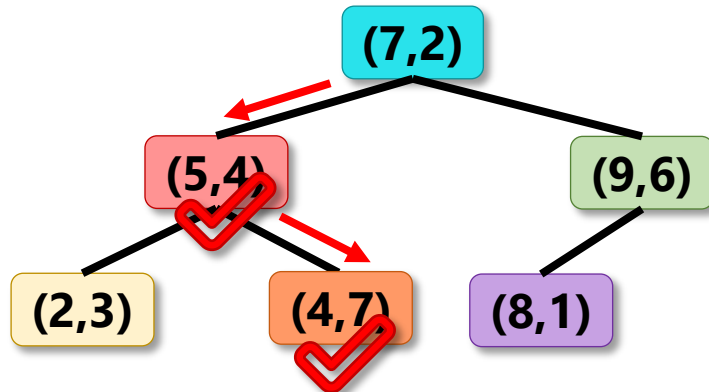
[input] constructed k-d tree; target point x ; **[output]** nearest neighbors of x .

- ① **Find the leaf node containing x in the k-d tree**
 - ① **Starting from the root node, recursive downward**, accessing the k-d tree
 - ② If the **coordinates** of the target point x in the current dimension are **smaller** than the coordinates of the cut-off point, move to the **left** child node, otherwise move to the right child node
 - ③ **Until** the node is a leaf node
- ② **Use this leaf node as the current nearest point and calculate the distance from the current nearest point to the target point**
- ③ **Recursively back up and perform the following operations at each node:**
 - ① If the node is closer to the target point x than the current nearest point, the node will be the "**current nearest point**"
 - ② Check whether there is **another child node** of the **parent node of the current nearest point**, and whether there is a **closer point** in the corresponding region. (the distance from the target point x to the current nearest point, compared with the vertical distance from x to the division line of the parent node)
 - a) **Intersection:** there may be a point closer to the target point in the region corresponding to another child node, **move to another child node and recursively perform a nearest neighbor search**
 - b) Non-intersection: **back up**.
- ④ **When back to the root node, the search ends and the current nearest neighbor is the nearest neighbor of x**

2.4 Which level of nodes needs to be calculated when backtracking?

? What to do when a new node that needs to be searched for nearest neighbors lands near the root node?

Let input point $x = (7, 6)^T$, we have to find the nearest $M = 2$ points. The red point shown in Figure below is the input point x



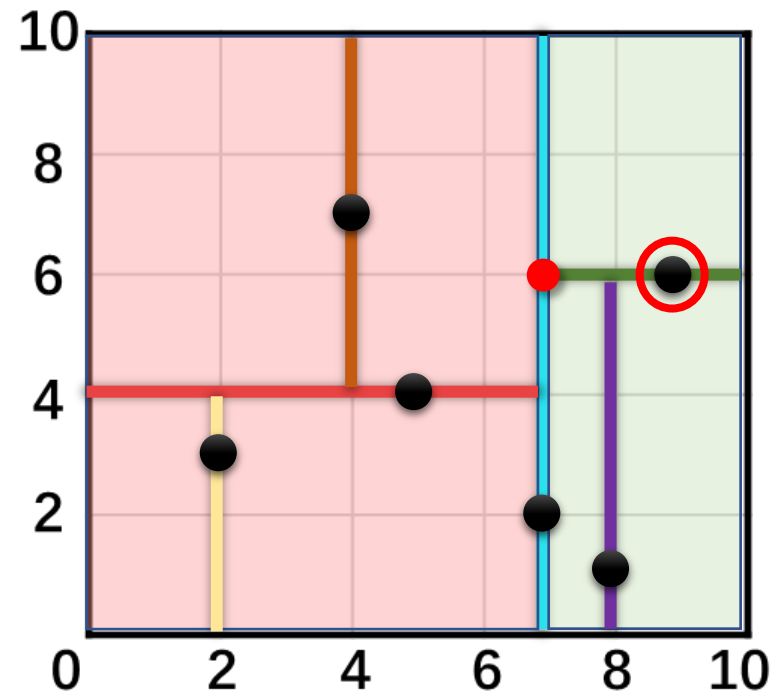
(1) Not bigger than 7, select the left subtree

(2) Bigger than 4, select the right subtree

(3) Record this current nearest point and calculate the distance from the target point to the current nearest point

(4) Check, check, check, when back to the root node, the search ends and the current nearest neighbor is the nearest neighbor of x

BUT, if we see the picture.....



Nearest Neighbor Set Q:

Q1 = [(4, 7)], $D = \sqrt{10}$

Q2 = [(5, 4)], $D = \sqrt{8}$

You will find that (9, 6) is **missing**, it is on the other half of the subtree of the root node

2.4 K-d Tree: Reflections



In the k-d tree, we query the **nearest neighbor of the target point x** . Then we first do a traversal down the tree to determine the smallest spatial region containing x . Then reverse the tree by using the split rule of k-d tree to **backtrack the tree**. In this process, we keep searching the other half of the parent node's children to check other cells that may contain x 's nearest neighbors. Obviously, **the number of nodes to be checked grows exponentially with increasing dimension k** .

It is shown that: for a k-d Tree (**k dimensional**) with **N nodes**, the time complexity of the **nearest neighbor search** is

$$t_{worst} = O(k \times N^{1-\frac{1}{k}})$$

Since a large number of backtracking can lead to a significant reduction in the k-d tree nearest neighbor search performances, researchers try to use **Best-Bin-First (BBF)**, which effectively reduces the number of backtracking by **setting a priority queue and running a timeout schedule** to obtain the **approximate** nearest neighbors.

2.5 Curse of Dimensionality

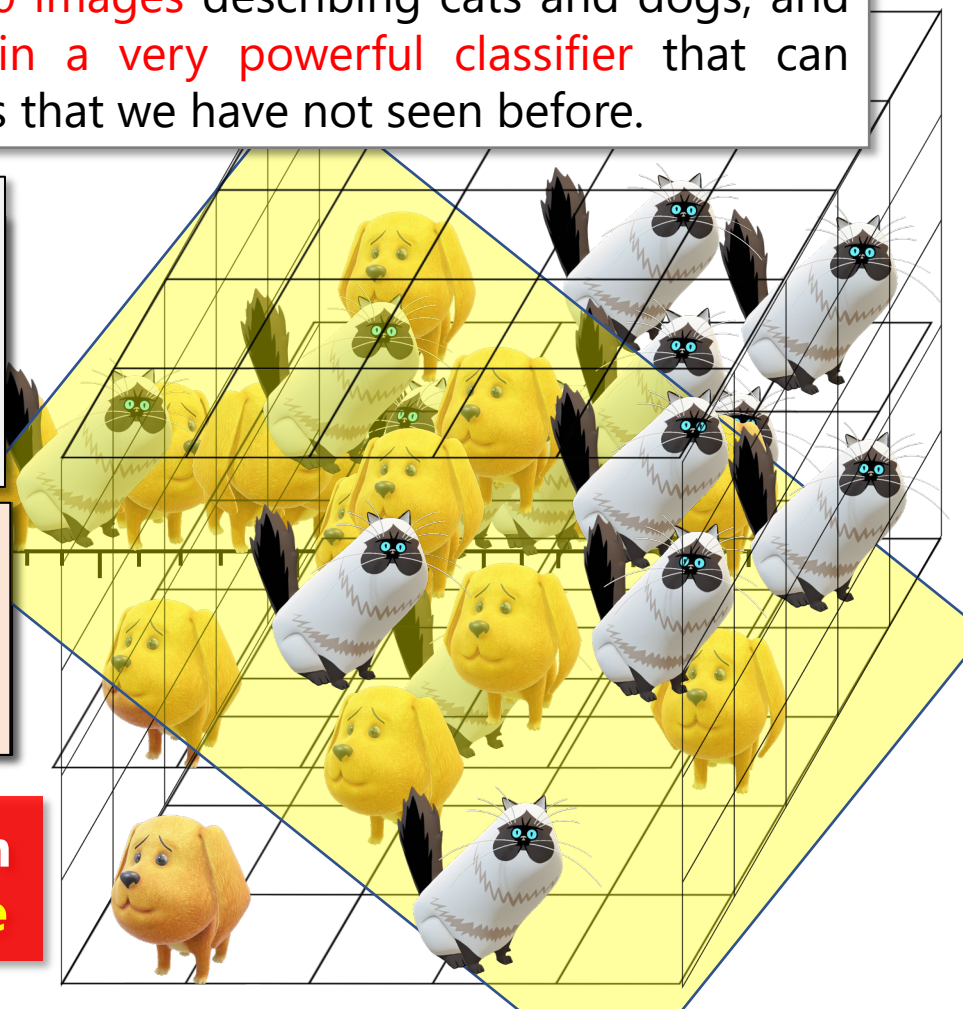
For most data, it is difficult to partition completely in one dimensional space or rather low dimensional space, but it is often possible to find a **hyperplane** in high latitude spaces that will partition it perfectly.

? Suppose we only have a total of **10 images** describing cats and dogs, and need to use these 10 images to **train a very powerful classifier** that can accurately identify various cats and dogs that we have not seen before.

After use 2 feature, we still cannot find a simple straight line to classify them effectively. For this reason, **we add one more feature**, "blue", expanding to a 3-dimensional feature space

In the 3-dimensional feature space, it is easy to find a classification plane that can effectively classify cats and dogs on the training set.

In **high dimensional space**, we seem to get **better classifier performance**



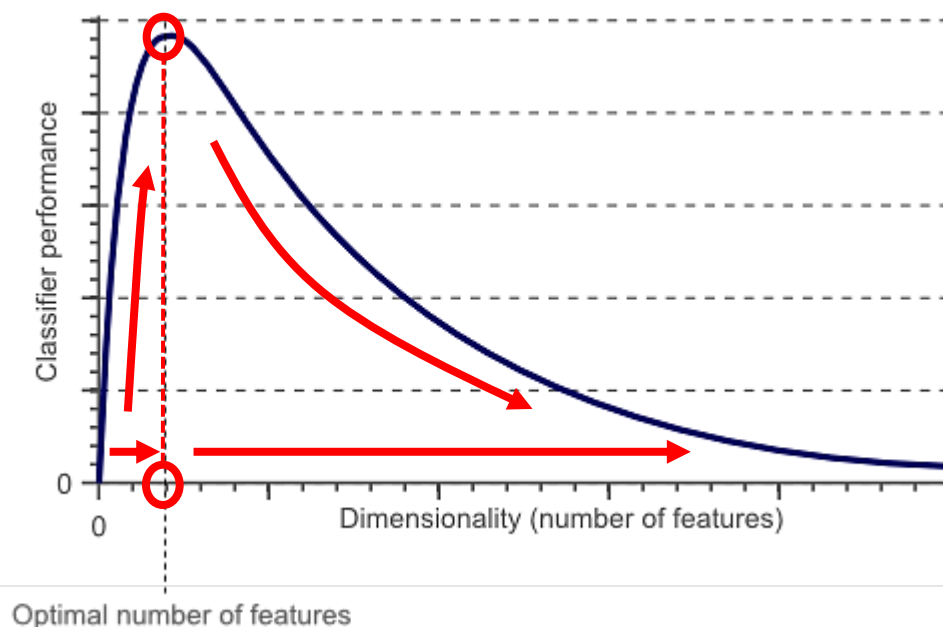
2.5

Curse of Dimensionality: Dimensionality vs Classifier performance

The above illustrations might seem to suggest that: **increasing the number of features until perfect classification results are obtained**—Is it the best way to train a classifier?

Curse of Dimensionality

In fact, **after a certain point, increasing the dimensionality of the problem by adding new features would actually degrade the performance of our classifier.** It is often referred to as '**The Curse of Dimensionality**'.



As the dimensionality increases, the classifier's performance increases until the optimal number of features is reached. Further increasing the dimensionality without increasing the number of training samples **results in a decrease in classifier performance.**

2.5

Curse of Dimensionality: the optimal number of **features**



Take **10 images of dogs and cats** as an example:

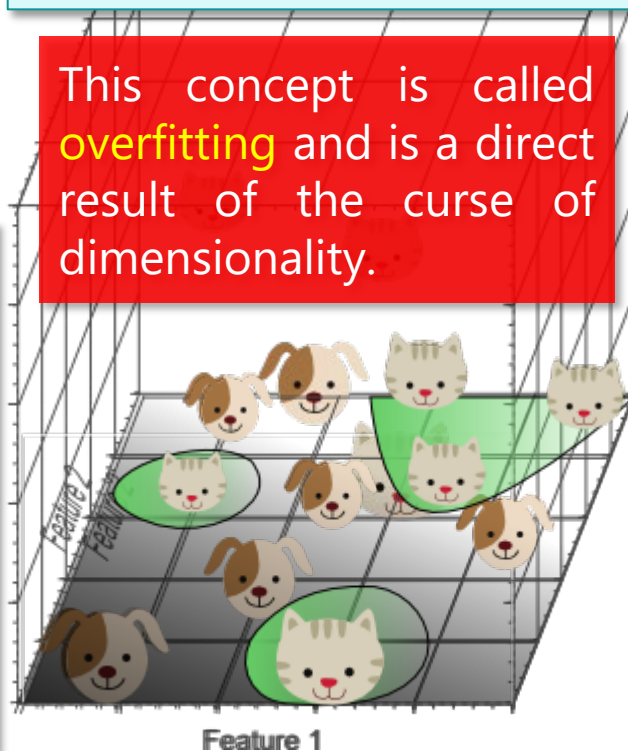
- 1) In the 1D case, 10 training instances covered the complete 1D feature space, the width of which was 5 unit intervals. The sample density was $10/5=2$.
- 2) In the 2D case, we still had 10 training, now they cover a 2D feature space with an area of $5 \times 5 = 25$ unit squares. The sample density was $10/25 = 0.4$.
- 3) In the 3D case, the 10 samples had to cover a feature space volume of $5 \times 5 \times 5 = 125$ unit cubes. The sample density was $10/125 = 0.08$.

If we project the highly dimensional classification result back to a lower dimensional space: adding the third dimension to obtain perfect classification results, simply corresponds to using a complicated non-linear classifier in the lower dimensional feature space. **The classifier learns the appearance of specific instances and exceptions of our training dataset and the resulting classifier would fail on real-world data.**

If we **don't increase samples and only increase features**, the **samples** will become **increasingly sparse** as the dimensionality grows.

The benefit is **easier to find a hyperplane to separate the targets**.

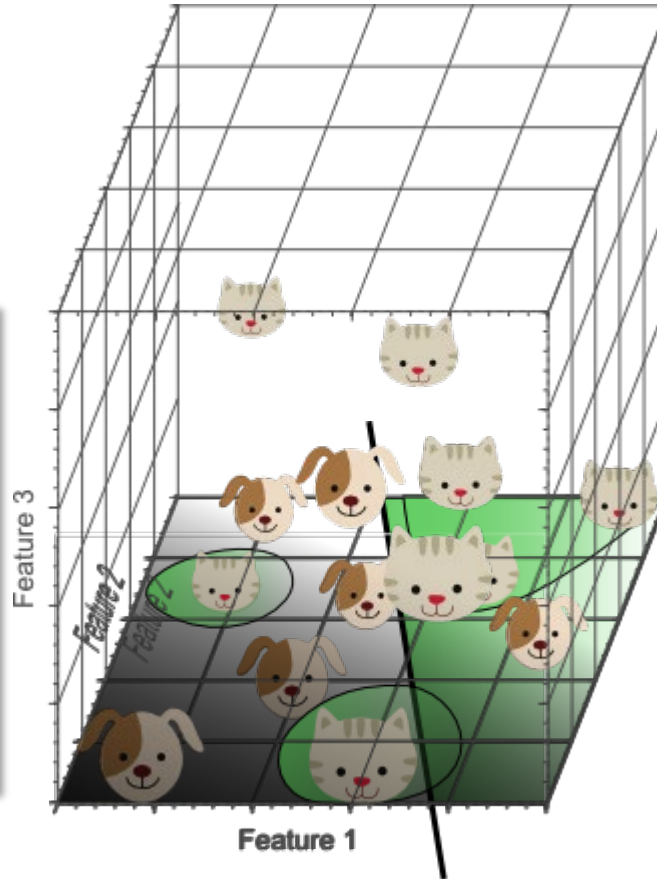
This concept is called **overfitting** and is a direct result of the curse of dimensionality.



2.5

Curse of Dimensionality: dimensionality reduction

Using **too many features** results in overfitting. The classifier starts **learning exceptions** that are specific to the training data and **do not generalize well** when new data is encountered.



Using **only 2 features instead of 3**. Although the training data is not classified perfectly, this simple linear classifier **with better generalization capabilities** (linear classifiers is not as sensitive to noise as nonlinear classifiers, avoiding curse of dimensionality by using fewer features, and avoiding overfitting in the high-dimensional case)

But: We often use linear classifiers in high-dimensional space and complex classifiers in low-dimensional space

2.5

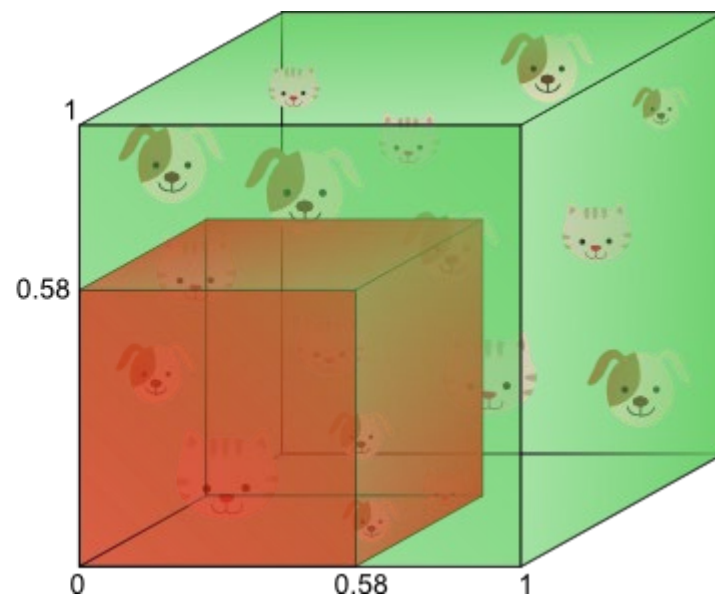
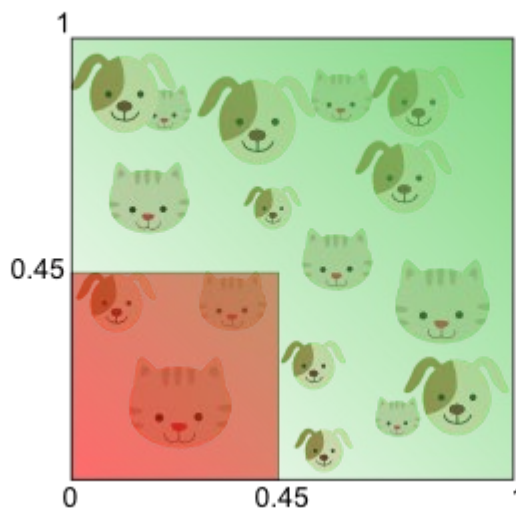
Curse of Dimensionality: the optimal number of training data



If we want to **train a classifier** using only a single feature whose value ranges from 0 to 1 (distributed), and this feature is unique for each cat and dog:

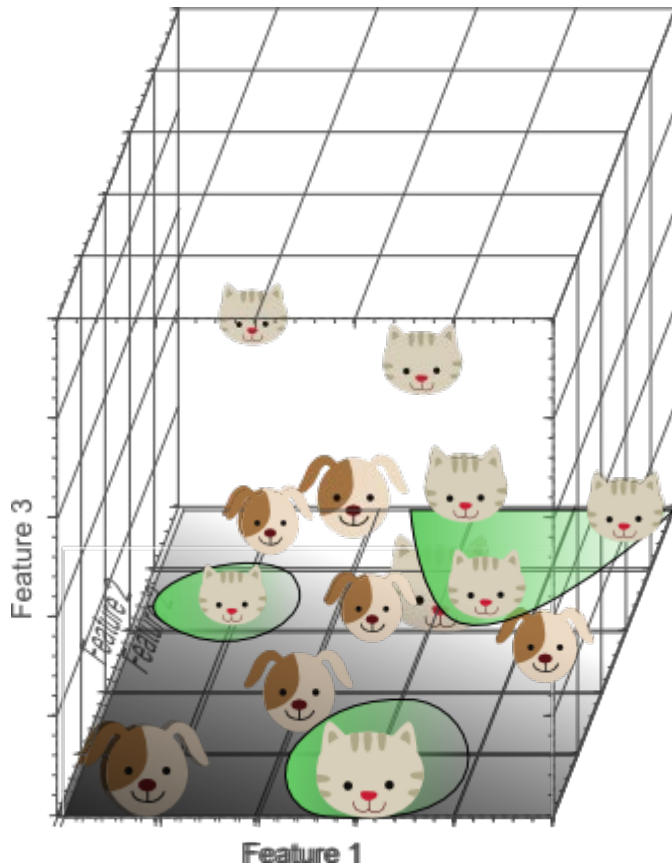
- 1) If we want our training data to cover 20% of this range, then the amount of training data needed is 20% of the complete population of cats and dogs.
- 2) if we add another feature → a 2D feature space, need to obtain 45% of the complete population of cats and dogs in each dimension ($0.45^2 = 0.2$).
- 3) In the 3D case, to cover 20% of the 3D feature range, need to obtain 58% of the population in each dimension ($0.58^3 = 0.2$).

If **increasing feature dimension**, as sample becomes increasingly **sparse**, we will **increase the number of samples at an exponential level** to avoid overfitting

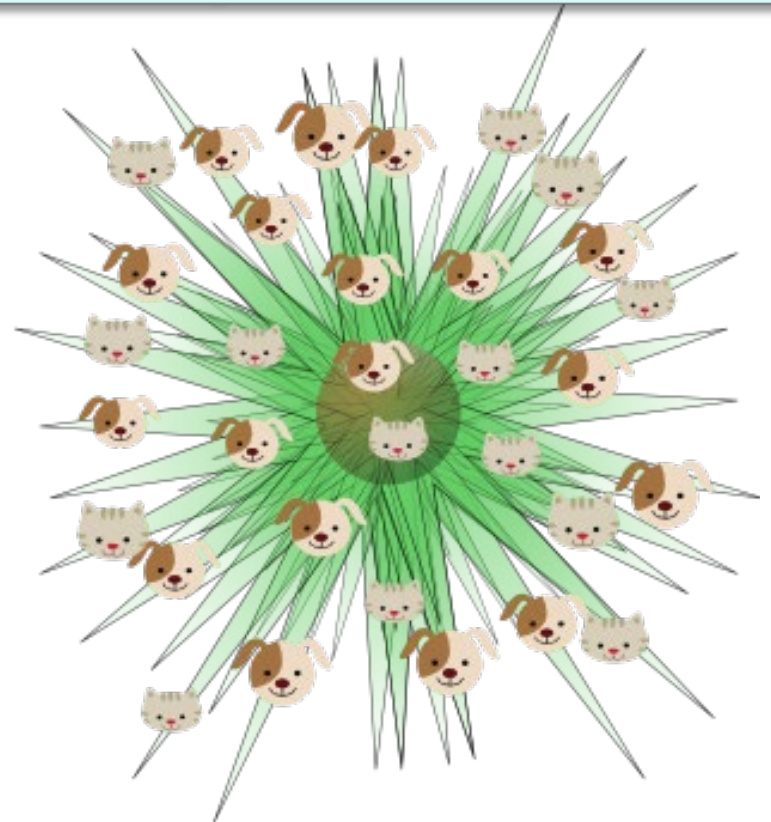


2.5 Curse of Dimensionality

On the one hand, if the amount of available training data is fixed, then **overfitting occurs** if we **keep adding dimensions**.



On the other hand, if we **keep adding dimensions**, the **amount of training data needs to grow exponentially** fast to maintain the same coverage and to **avoid overfitting**.



2.5

Curse of Dimensionality: dimensionality reduction

A: What could we do (to reduce the curse of dimensionality)?

B: Desperately increasing the amount of training data!

A: Don't be silly, exponential growth, you don't have that many resources

B: Must the feature dimension of the data be large?

A: The **natural representation of data is often high-dimensional**, such as the document check we mentioned before, and product recommendations and so on

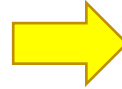
B: We tend to represent data with a large number of dimensions, but **need a small number of dimensions** in order to ensure computational tractability

B: It seems that we can only use **dimensionality reduction**

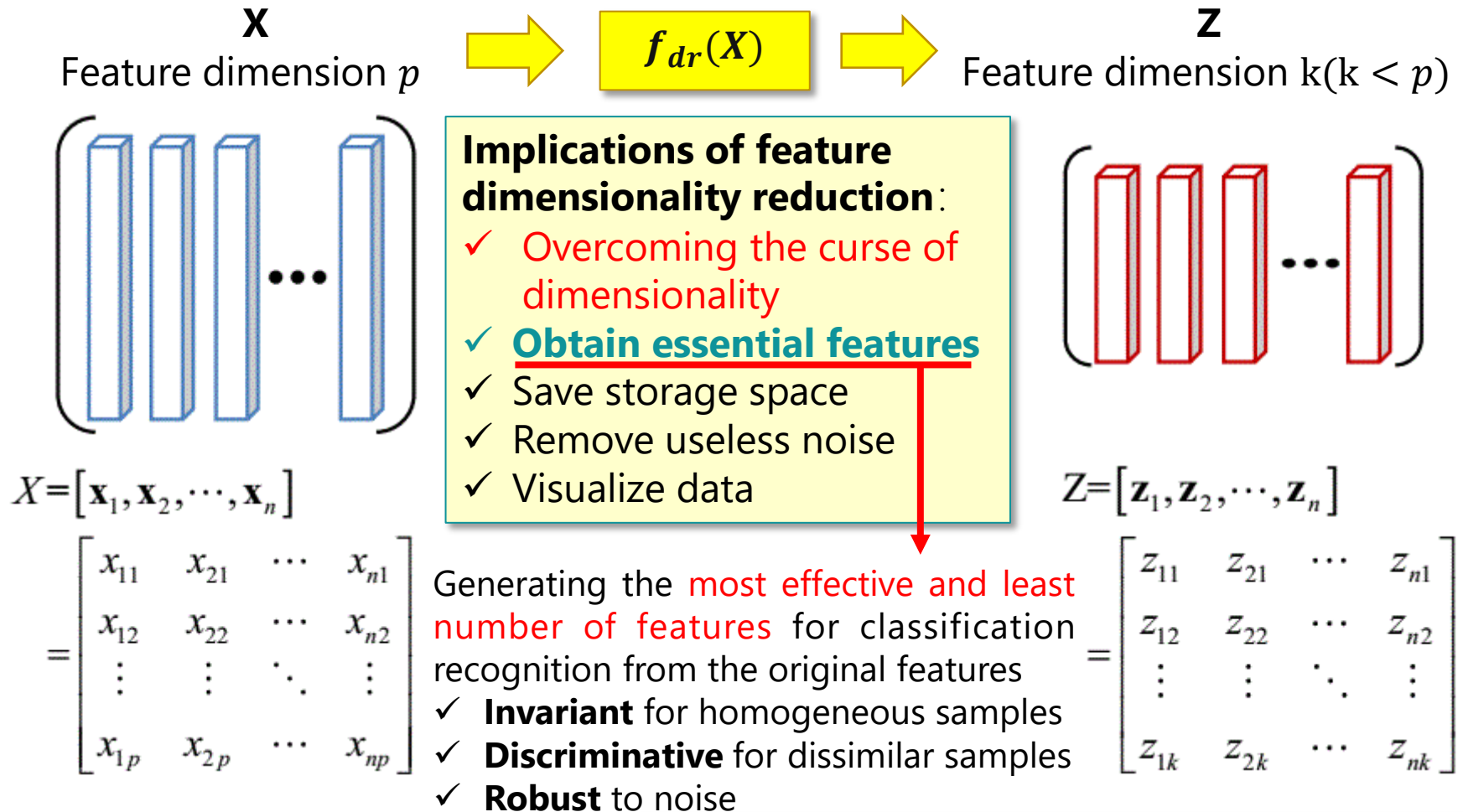
A: In fact, there is no fixed rule to determine the "too large" feature dimension, it depends on the **number of training samples**, the **complexity of the classification boundary shape**, and the **type of classifier**

2.5 Curse of Dimensionality: dimensionality reduction

The Curse of Dimensionality



Dimensionality Reduction



Common methods for dimensionality reduction.

□ Feature selection

□ Feature transformation

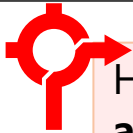
2.6 Minhash: from Jaccard

?

One of the more fundamental problems in data mining is to **compare the similarity of two sets**.

Dumbest way

use a two-fold loop to iterate through all the elements in the 2 sets, and then count the number of identical elements in the 2 sets.



However, when **the number of elements in these two sets is very large** and **there are many set-pairs have to calculate the similarity at the same time**, the consumption of memory and time is very **large**.

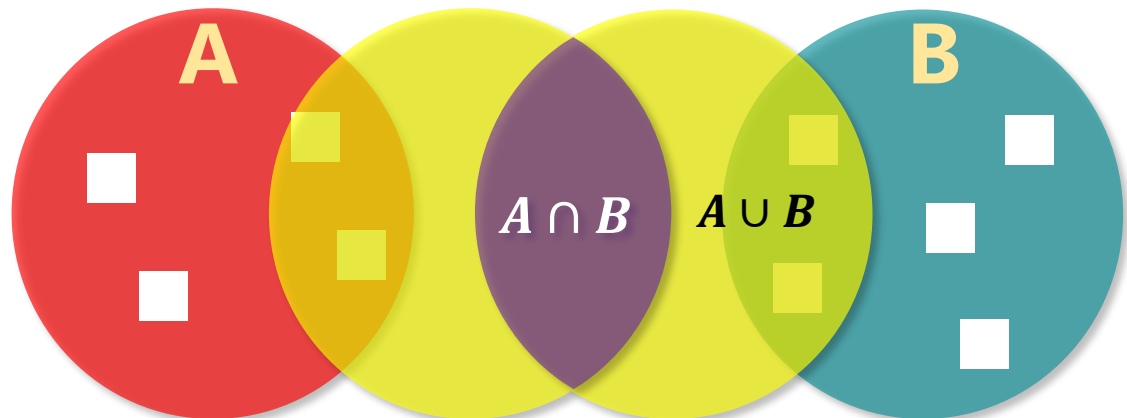
Recall

from this lecture we have introduced a method to compare the similarity between two sets - **Jaccard**

Suppose there are two sets A, B. We denote the Jaccard similarity of sets A and B as $SIM(A, B)$, then

$$J(A, B) = SIM(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

$$SIM(A, B) = \frac{2}{7}$$



2.6 Minhash: Feature Matrix

Suppose there are 4 sets, S_1, S_2, S_3, S_4 .

$$S_1 = \{a, d\}$$

$$S_2 = \{c\}$$

$$S_3 = \{b, d, e\}$$

$$S_4 = \{a, c, d\}$$

The full set $U = \{a, b, c, d, e\}$

No.	Item	S_1	S_2	S_3	S_4
0	a	1	0	0	1
1	b	0	0	1	0
2	c	0	1	0	1
3	d	1	0	1	1
4	e	0	0	1	0

U Feature Matrix

Feature Matrix

A **column** of the feature matrix corresponds to a **set**, and all the **rows add up to the full set** of all set elements, If the set has that element, the corresponding position in the matrix is **1**, otherwise it is **0**.

The feature matrix of the set is constructed to **calculate the minHash of the set**

2.6 Minhash: calculate the minHash

1. First upset the rows of the feature matrix

No.	Item	S_1	S_2	S_3	S_4
0	<i>a</i>	1	0	0	1
1	<i>b</i>	0	0	1	0
2	<i>c</i>	0	1	0	1
3	<i>d</i>	1	0	1	1
4	<i>e</i>	0	0	1	0



No.	Item	S_1	S_2	S_3	S_4
0	<i>b</i>	0	0	1	0
1	<i>e</i>	0	0	1	0
2	<i>a</i>	1	0	0	1
3	<i>d</i>	1	0	1	1
4	<i>c</i>	0	1	0	1

Randomly transpose the position between rows

2. The **minimum hash value of a column** is equal to the row number of the first row with a value of 1 in that column after disruption, and the row number starts from 0.

For column S_1 , go down from the first row of this column until you encounter the first 1. And the row number where it is located is the minimum hash value of this column.

So the minimum hash values of these 4 columns:

$$h(S_1) = 2$$

$$h(S_2) = 4$$

$$h(S_3) = 0$$

$$h(S_4) = 2$$



●			●
		●	
	●		●
●		●	●
		●	

No.	No.	Item	S_1	S_2	S_3	S_4
2	0	a	1	0	0	1
0	1	b	0	0	1	0
4	2	c	0	1	0	1
3	3	d	1	0	1	1
1	4	e	0	0	1	0

Swap the drawers (letters); the balls in the drawers (indicating that the set contains those letters) move with the drawers. After swapping, the letters contained in the set remain unchanged, but the letter indices change, and thus the minimum hash value of the set changes.



		●	
		●	
●			●
●		●	●
	●		●

No.	Item	S_1	S_2	S_3	S_4
0	b	0	0	1	0
1	e	0	0	1	0
2	a	1	0	0	1
3	d	1	0	1	1
4	c	0	1	0	1

Since the balls move with the drawers, we can also interpret this as renumbering the drawers (see the red numbers).

2.6 Minhash: minHash vs. Jaccard

After row disruption, the **probability** that the **minHash values of two sets are equal** **=** **Jaccard similarity** of these two sets

Suppose only the sets S_1 and S_2 are considered, then there are three types:

1. This row: **both** S_1 and S_2 has a value of **1** → noted as **class X**.

2. This row: **Only one set** has a value of **1** → noted as **class Y**.

3. This row: **both** S_1 and S_2 has a value of **0** → noted as **class Z**.

No.	Item	S_1	S_2
0	<i>b</i>	0	0
1	<i>e</i>	0	0
2	<i>a</i>	1	1
3	<i>d</i>	1	0
4	<i>c</i>	0	0

Suppose there are x rows belonging to class **X** and y rows belonging to class **Y**

$$\text{So } S_1 \cap S_2 = x, S_1 \cup S_2 = x + y$$

$$\text{SIM}(S_1, S_2) = J(S_1, S_2) = \frac{x}{x + y}$$

However, we tend to **ignore** $N_{A_0B_0}$ when $N_{A_0B_0}$ is large or when we do not focus on the case it represents

After row disruption, the probability of encountering a row of class **X** before encountering a row of class **Y** is $x / (x + y)$ when the feature matrix is scanned from top to bottom; And since $h(S_1) = h(S_2)$ in the class **X** row, the probability that $h(S_1) = h(S_2) = x / (x + y)$

$$\rho_{h(S_1)=h(S_2)} = \frac{x}{x + y}$$

Understanding Why MinHash Collision Probability = Jaccard Similarity

1. Build Intuition First: A "Lottery" Analogy

Imagine every element in the universal set U is a lottery ticket. Now there are two players, $S1$ and $S2$, each holding some tickets (i.e., their respective element sets).

Game rules: Shuffle all tickets randomly, then flip through them from top to bottom. Whoever flips their own ticket first wins, and that ticket's number is their MinHash value.

Question: What is the probability that both players win with the **same** ticket (i.e., their MinHash values are equal)?

Answer: It equals the number of tickets they **both** hold $\div$ the total number of tickets held by **either** player — that's exactly the **Jaccard similarity**!

2. Rigorous Proof: Three Types of Rows

Let's use a concrete example. Let $U = \{a, b, c, d, e\}$, with two sets:

Row (Element)	S1	S2	Type
a	1	0	Y (only S1 has it)
b	0	0	Z (neither has it)
c	0	1	Y (only S2 has it)
d	1	1	X (both have it)
e	0	0	Z (neither has it)

Jaccard similarity:

$$J(S1, S2) = \frac{|S \cap T|}{|S \cup T|} = \frac{x}{x+y} = \frac{1}{3}$$

Rows are divided into three types:

Type	Meaning	Count
X	Both columns are 1 (intersection)	$x=1$ (element d)
Y	One column is 1, the other is 0 (exclusive to union)	$y=2$ (elements a, c)
Z	Both columns are 0	$z=2$ (can be ignored)

3. Core Derivation: $P(h(S_1) = h(S_2)) = \frac{x}{x+y}$

After randomly permuting the rows of the matrix, we scan from top to bottom, taking the row index of the **first 1** in each column as the MinHash value.

(1) Key Observation

$h(S_1) = h(S_2)$ if and only if: we encounter an **X-type** row **before** any **Y-type** row.

● Why?

Case	Scanning Process	Result
✓ Hit X-type row first	Both columns are 1 at this row → both MinHash values equal this row index	Equal
✗ Hit Y-type row first	Suppose $S_1=1, S_2=0$ at this row → $h(S_1)$ is fixed, S_2 keeps searching and must find a different row	Not equal

*Z-type rows (both 0) contribute nothing to either player — just skip them, no effect on the result.

(2) Probability Calculation

After random shuffling, all $x + y$ "effective rows" (X and Y types) are equally likely to appear in any order.

Question: What is the **probability** that the **first** effective row to appear is of **type X**?

$P(\text{hit X first}) = \frac{x}{x+y}$ This is exactly the Jaccard similarity!

2.6 Minhash: minHash Signature

$$S_1 = \{a, d\}$$

$$S_2 = \{c\}$$

$$S_3 = \{b, d, e\}$$

$$S_4 = \{a, c, d\}$$

The full set $U = \{a, b, c, d, e\}$

No.	Item	S_1	S_2	S_3	S_4	
0	<i>b</i>	0	0	1	0	$h(S_1) = 2$
1	<i>e</i>	0	0	1	0	$h(S_2) = 4$
2	<i>a</i>	1	0	0	1	$h(S_3) = 0$
3	<i>d</i>	1	0	1	1	$h(S_4) = 2$
4	<i>c</i>	0	1	0	1	

When we use a row disruption to deal with the feature matrix, we can get the minHash value of each set, so that multiple sets will have multiple minHash values

No.	Item	S_1	S_2	S_3	S_4	
0	<i>d</i>	1	0	1	1	$h(S_1) = 0$
1	<i>b</i>	0	0	1	0	$h(S_2) = 2$
2	<i>c</i>	0	1	0	1	$h(S_3) = 0$
3	<i>a</i>	1	0	0	1	$h(S_4) = 0$
4	<i>e</i>	0	0	1	0	

No.	Item	S_1	S_2	S_3	S_4	
0	<i>e</i>	0	0	1	0	$h(S_1) = 1$
1	<i>a</i>	1	0	0	1	$h(S_2) = 2$
2	<i>c</i>	0	1	0	1	$h(S_3) = 0$
3	<i>d</i>	1	0	1	1	$h(S_4) = 1$
4	<i>b</i>	0	0	1	0	

When we use n row disruption to deal with the feature matrix, calculate the minHash value of n matrices; for the set S_i , there will be n minhash values, and these n hash values can form a column vector

Signature matrix of the feature matrix

If there are m sets, there will be m column vectors with n elements (n row disruption) in each column vector. Usually, n will be much smaller than the number of rows of the feature matrix, so the signature matrix will be much smaller than the feature matrix.

When a feature matrix is very large (suppose there are hundreds of millions of rows), it is very timely to row disruption for several times, and we can actually simulate the effect of row disruption by some random hash functions.

2.6 Minhash: Calculation of the minimum signature

We can actually simulate the effect of row disruption by some random hash functions



Using the **hash function**, a hash transform is performed on the row No.

No.	S_1	S_2	S_3	S_4	$h_1: x + 1 \bmod 5$	$h_2: 3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

No.	Item	S_1	S_2	S_3	S_4
0	<i>a</i>	1	0	0	1
1	<i>b</i>	0	0	1	0
2	<i>c</i>	0	1	0	1
3	<i>d</i>	1	0	1	1
4	<i>e</i>	0	0	1	0

For 0,1,2,3,4, the mapping can be done by $h(x) = (x+1) \bmod 5$ to get 1,2,3,4,0.

For example, when $n=5$, for 0,1,2,3,4, the mapping can be done by $h(x) = (3x+1) \bmod 5$ to get 1,4,2,0,3.

Of course, random hash functions like $h(x) = ax + b$ may obviously conflict, but as long as n and a are coprime, then the generated must be an alignment, which is well proven with congruence classes

2.6 Minhash: Calculation of the Signature Matrix

We have generated two new sets of row numbers using the h_1 and h_2 hash functions

No.	S_1	S_2	S_3	S_4	$h_1: x + 1 \bmod 5$	$h_2: 3x + 1 \bmod 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

1. All initialized to **inf (infinity)**

Find the value of S_1

Iterate through the corresponding words of S_1 , from line 0 to line 4.

1) Row 0 is 1 in S_1 , look at the line number calculated by h_1 as $1 < \text{inf}$, assign h_1 as 1 (which is the line number). Continue traversing

2) Row 1 is 0, don't care, skip

3) Row 2 is 0, don't care, skip

4) Row 3 is 1, look at the line number calculated by h_1 is 4. 4 is greater than the value of h_1 at this time, the value of h_1 remains unchanged. If it is less than the value of h_1 at this point, the value is paid to h_1

5) Row 4 is 0, don't care, skip

\	S_1	S_2	S_3	S_4
h_1	inf	inf	inf	inf
h_2	inf	inf	inf	inf

Find the **smallest line number** that S_1 value is not 0 as the value of h_1 output

\	S_1	S_2	S_3	S_4
h_1	1	3	0	1
h_2	0	2	0	0

1. Does minhash achieve feature dimensionality reduction?

2. $\text{SIM}(\mathbf{S}_1, \mathbf{S}_3)$ 、 $\text{SIM}(\mathbf{S}_1, \mathbf{S}_4)$ 、 $\text{SIM}(\mathbf{S}_3, \mathbf{S}_4)$?
3. Direct calculation of the Jaccard $(\mathbf{S}_1, \mathbf{S}_3)$
 $\text{Jaccard}(\mathbf{S}_1, \mathbf{S}_4)$ $\text{Jaccard}(\mathbf{S}_3, \mathbf{S}_4)$ of the original set?

$\backslash$	$\mathbf{S}_1$	$\mathbf{S}_2$	$\mathbf{S}_3$	$\mathbf{S}_4$
h_1	1	3	0	1
h_2	0	2	0	0

2. $\text{SIM}(\mathbf{S}_1, \mathbf{S}_3)$: the **probability** that the minHash values of two sets $(\mathbf{S}_1, \mathbf{S}_3)$ are **equal**

$$h_1(\mathbf{S}_1) = 1, h_1(\mathbf{S}_3) = 0 \quad \text{Not equal}$$

$$h_2(\mathbf{S}_1) = 0, h_2(\mathbf{S}_3) = 0 \quad \text{equal}$$

$\text{SIM}(\mathbf{S}_1, \mathbf{S}_3) = \frac{1}{2}$ (After 2 random permutations, the minhash values of two sets are equal once, so the probability of equality is 1/2)

$$\text{SIM}(\mathbf{S}_1, \mathbf{S}_4) = 2/2 = 1$$

$$\text{SIM}(\mathbf{S}_3, \mathbf{S}_4) = 1/2$$

3. Jaccard $(\mathbf{S}_1, \mathbf{S}_3) = \frac{1}{4}$

Theoretically, the probability that MinHash values are equal equals the Jaccard similarity; in practice, when performing a single comparison or **estimation with a finite number of hash functions**, any observed **inequality** is due to normal random error or estimation error, not a theoretical contradiction.

Research Assignment:

Submit the report to luoxsince1989@qq.com by **May 29**.

Topic: Johnson–Lindenstrauss Lemma (JL Lemma)

Requirements:

1. List the papers you have studied (at least 3).

2. Understand the core idea and mathematical formulation of the JL Lemma.

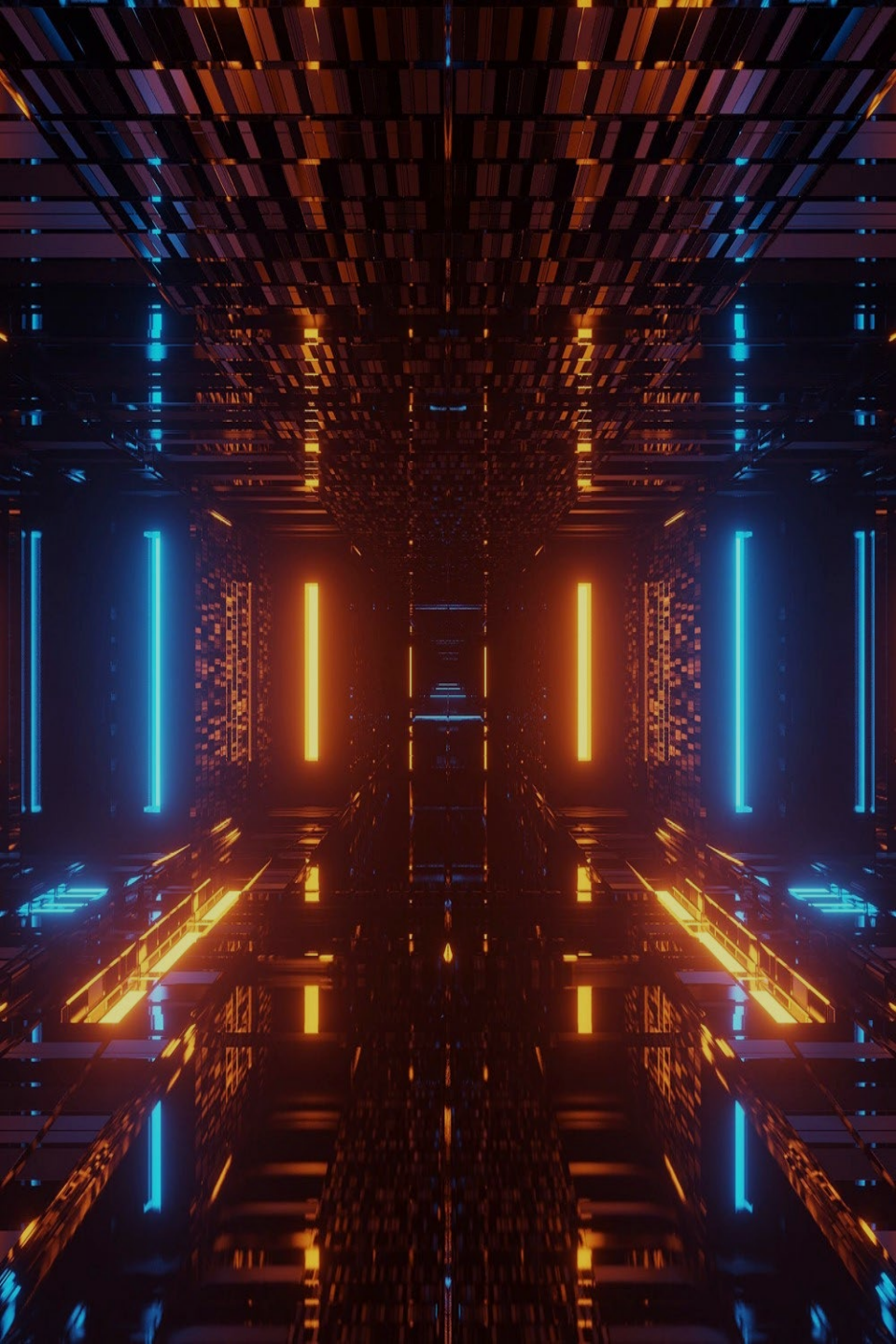
Restate the lemma using precise mathematical language, clarify the relationship among the parameters ε, n, k and so on, and analyze the difference between **approximate preservation of Euclidean distance and that of inner product**.

3. Compare two common constructions of random projection matrices (e.g., Gaussian random matrix, binary-coin matrix (Achlioptas construction), sparse random matrix, etc.) Analyze their **respective advantages** and **suitable application scenarios**.

4. Investigate the applications of the JL Lemma in big data, such as dimensionality reduction preprocessing, approximate nearest neighbor search, and compressed sensing.

Select **one** application scenario and provide an analysis of **no less than 1000 words**.

5. The entire research report shall be no less than 3000 words.



Thanks